> **Referencia en vivo y laboratorio:** para el recorrido sobre el asistente desplegado y el laboratorio PostgreSQL/pgvector (`lab/`), consulta también `recorrido-cloud.md` . Este guion articula los 90 minutos de S10 sincronizados con las 15 diapositivas de `material/presentaciones/S10-rag-basico.pptx` .

Guion S10 · RAG básico: construcción, recorrido semántico y laboratorio

---

Fecha: jueves 22 de octubre de 2026 · **Cohorte:** M/J · **Horario:** 18:30–20:00 `Europe/Madrid` (90 minutos con preguntas).

Presentación: `material/presentaciones/S10-rag-basico.pptx` (`material/presentaciones/S10-rag-basico.pdf`).

Laboratorio del ejercicio: `lab/` (`bun run lab:rebuild` , `bun run lab:demo` , `bun run lab:eval`).

Demostrador del curso: `bun run dev` (respaldo local `demo`) / `https://rag-thepower.josenuevo.dev` .

Antes de empezar la clase

Sigue `material/notas/checklist-preclase.md` :

1. Comprueba `bun run preflight` (modo local `demo` sin gasto) y ten probado `bun run lab:demo` .
2. Deja abierta la presentación `S10-rag-basico.pptx` en la diapositiva 1, el navegador con el demostrador y una terminal en `/Users/jose/Documents/the-power/3-rag` para mostrar `lab/` .
3. Identidades clave para responder dudas:

- **Demostrador del curso (`apps/`):** corpus `c3a59a406888b` (22 documentos S1–S20 y calendarios L/X y M/J, 69 fragmentos); perfil real `p76509a5f9743` (`@cf/baai/bge-m3` , `1024` dimensiones, métrica `cosine` , umbral `0,50` , modelo `claude-haiku-5-5`); perfil offline `p9e8e8e03627e` (`fixture-token-hash-v1` , `128` d, umbral `0,33`).
- **Laboratorio del ejercicio (`lab/`):** corpus `lab-689164ca81cd` (10 documentos ficticios de AetherCloud, 20 fragmentos, `900` caracteres y `80` de solapamiento); esquema `lab/schema.sql` sobre PostgreSQL + `pgvector` con columna `vector(768)` e índice HNSW (`m = 16, ef_construction = 64`), umbral calibrado en validación `similitud ⩾ 0,25` (`distancia coseno ⟺ ⩽ 0,75`).

0–10 min · Objetivo, QR de S9 y pregunta sin documentos (Diapositivas 1 a 3)

Diapositiva 1 · RAG básico: Asistente del curso

Explica:

> Hoy construimos y recorremos un sistema RAG completo de principio a fin: cómo pasar de documentos Markdown en bruto a fragmentos con solapamiento, embeddings, un índice vectorial HNSW, una búsqueda que filtra por puntuación **antes** de llamar al modelo, citas verificables y un rechazo limpio cuando falta evidencia. Veremos el demostrador del curso AI

> Engineer (S1–S20) y el laboratorio reproducible con PostgreSQL y `pgvector` (`vector(768)`) en la carpeta `lab/`.

Diapositiva 2 · Antes de empezar (QR)

Deja el QR en pantalla unos segundos:

> Este QR evalúa la clase anterior: **S9 (PostgreSQL y pgvector, del martes 20 de octubre)**. Hoy damos continuidad directa a S9 conectando esa base vectorial con la canalización completa de RAG.

Diapositiva 3 · La misma pregunta, sin documentos

Pasa a la interfaz en la etapa **Sin RAG**, grupo **General**:

1. Escribe: `¿Cómo modela LangGraph un agente con estado y nodos?` o pregunta por la fecha exacta de S10 en M/J.
2. Señala que sin documentos recuperados no hay citas verificables, que una abstención o respuesta genérica es el comportamiento honesto, y que los campos de uso y coste no medidos viajan como `null` (`no medido`), nunca como `0`.

10–25 min · Construcción del índice: corpus, fragmentación, embeddings y HNSW (Diapositivas 4 a 6)

Diapositiva 4 · Corpus y perfiles del sistema

Muestra la tabla comparativa de la diapositiva 4:

- En el demostrador del curso: `c3a59a406888b` (22 fichas reales del programa y dos calendarios, 69 fragmentos) e índice real `p76509a5f9743` (`BGE-M3`, 1024 d, coseno `0,50`).
- En el laboratorio del ejercicio (`lab/`): `lab-689164ca81cd` con **10 documentos ficticios de AetherCloud** (arquitectura, mTLS, cuotas, cifrado KMS, SLA, incidentes, facturación, RGPD, OpenTelemetry y despliegues canary), reconstruibles desde cero sobre `vector(768)` + HNSW.

Diapositiva 5 · Fragmentación y solapamiento (`packages/rag/src/chunk.ts` y `lab/src/corpus.ts`)

Abre `packages/rag/src/chunk.ts` y explica las decisiones de construcción que el alumno aplicará en su práctica:

1. **Limpieza previa (`cleanDocumentText`):** normalizar saltos de línea, eliminar caracteres de control y limpiar espacios sin alterar cifras ni identificadores técnicos.
2. **Corte jerárquico por secciones (`##`) y párrafos:** nunca partir un documento por bytes ciegos ignorando la estructura; cada fragmento conserva el título del documento y el encabezado `##` de su sección.
3. **Tamaño (`maxChars: 900`) y solapamiento (`overlapChars: 80`):** explica por qué el solapamiento de 80 caracteres evita que una condición o cifra quede partida entre el final del fragmento `00` y el inicio del `01`.
4. **Identificador determinista (`chunkId`):** `<version>:<documentId>:<ordinal>` (p. ej. `lab-689164ca81cd:04-almacenamiento-y-cifrado-kms:00`). Si el contenido no cambia, el ID no cambia al reconstruir el índice.

Diapositiva 6 · Funcionamiento del embedding e índice vectorial (`lab/schema.sql`)

Abre `lab/schema.sql` y señala:

```
CREATE EXTENSION IF NOT EXISTS vector;
-- Columna de 768 dimensiones exigida por el ejercicio
embedding vector(768) NOT NULL
-- Índice HNSW para distancia coseno
CREATE INDEX IF NOT EXISTS lab_chunks_embedding_hnsw_idx
  ON lab_chunks USING hnsw (embedding vector_cosine_ops)
  WITH (m = 16, ef_construction = 64);
```

Explica tres invariantes esenciales:

- **Mismo modelo y misma dimensión en ingesta y consulta:** si indexas con `EMBEDDING_MODEL_ID` a `768` dimensiones, la consulta debe usar exactamente ese modelo y `768` dimensiones (`assertVector768`). Mezclar familias o dimensiones invalida las distancias.
- **Qué hace el índice HNSW (`m = 16, ef_construction = 64`):** construye un grafo navegable multicapa para aproximar los vecinos más cercanos sin escanear toda la tabla en corpus grandes.
- **Escala en `pgvector` :** el operador `⟺` devuelve **distancia coseno** en \([0, 2]\), donde **menor significa más próximo** (`0` = idénticos). La similitud coseno es `1 - (embedding ⟺ $1::vector(768))` .

25–40 min · Recorrido semántico: consulta, filtro y calibración del umbral (Diapositivas 7 y 8)

Diapositiva 7 · Consultar, filtrar y puntuar (`apps/api/src/app.ts:670` y `lab/src/rag-basic.ts`)

Recorre los cinco pasos de la búsqueda semántica:

1. **Embeber la pregunta** con el mismo modelo del índice.
2. **Filtrar en la consulta a la base de datos** (por versión de corpus, modelo y grupo/audiencia), no después en el prompt.
3. **Ordenar por distancia/similitud** y recuperar `topK = 4` candidatos.
4. **Aplicar el umbral calibrado antes de llamar al modelo generativo.**
5. **Construir el contexto** solo con los fragmentos que superaron el corte.

En el navegador, cambia a **RAG básico**, grupo **General**, y pregunta: `¿Cuándo es la clase de RAG?` . Muestra que pide elegir grupo (`L/X` o `M/J`) sin gastar llamadas.

Diapositiva 8 · Calibración del umbral y rechazo previo

Explica cómo se elige un umbral de forma rigurosa:

- **Conjunto de validación separado del test:** en `lab/src/evaluation.ts` (`LAB_VALIDATION_SET` , 6 casos que no están en las 20 preguntas de test), la pregunta sin respuesta más cercana dio `similitud = 0,148` (`distancia = 0,852`) y la respondible más baja dio `similitud = 0,250` (`distancia = 0,750`). Por eso el umbral del laboratorio queda en **`similitud ⩾ 0,25` (`distancia ⩽ 0,75`)**. En el demostrador con BGE-M3 (`scripts/calibrate-live.ts`), el corte calibrado es **`0,50`** .

- **Un prompt de «si no sabes, dilo» no sustituye este control:** si ningún fragmento supera el umbral, el código devuelve el rechazo directamente (`generationCalled: false`) sin invocar al LLM.

40–60 min · Construcción del contexto, citas verificables y variante `complete` (Diapositiva 9)

Diapositiva 9 · Construcción del contexto y citas

Pregunta en **RAG básico**: `¿Cómo modela LangGraph un agente con estado y nodos?` (y ejecuta en terminal `bun run lab:demo` para ver el caso equivalente de cifrado `AES-256-GCM` y enfriamiento de `72 horas` en `lab/`):

1. **Bloque `DATOS` no fiable (`prompts.ts` y `buildUntrustedDataBlock` en `lab/src/rag-basic.ts`):** enseña cómo los fragmentos seleccionados se empaquetan dentro de etiquetas de datos advirtiendo al modelo de que su contenido nunca puede reconfigurar las instrucciones del sistema.
2. **Validación de citas (`suppressUnknownCitations`):** el servidor comprueba cada marca `[cite:id]` contra el conjunto de fragmentos realmente seleccionados; si el modelo inventa un ID que no estaba en el contexto, se suprime.
3. **Variante de contexto completo (`RAG_CONTEXT_MODE=complete` , `apps/api/src/course-context.ts`):**

- Después de haber enseñado el recorrido semántico principal, explica la variante `complete` : como el corpus del curso tiene solo 23 000 caracteres (69 fragmentos), cabe entero en D1 sin truncar (tope de 200 fragmentos y 32 000 caracteres).
- Recalca la diferencia: `complete` evita que una pregunta de calendario con coseno `0,446 < 0,50` se quede fuera en un corpus diminuto, pero **no cumple ni demuestra el requisito del ejercicio de seleccionar por puntuación y decidir antes de generar**. Por eso la referencia principal de S10 y del laboratorio es el recorrido semántico.

60–75 min · Rechazo previo, cohortes, reconstrucción e ingesta desde cero (Diapositivas 10 y 11)

Diapositiva 10 · Si no consta, se dice (rechazo previo y cohortes)

1. **Rechazo por el control previo:** pregunta `¿Qué beca interna del año 1991 cubre este programa?` (y en `lab:demo` observa la consulta por licencias de Windows Server 2025). Señala en la traza que la decisión es `refuse` , sin fuentes y **sin llamada al modelo generativo**.
2. **Aislamiento de cohortes:** selecciona el grupo de **martes y jueves (`M/J`)** y pregunta `¿Cuándo es la clase de RAG S10?` . Cita `c3a59a406888b:calendario-mj:10` (`2026-10-22`) y no mezcla la fecha de `L/X` (`2026-10-26`).

Diapositiva 11 · Reconstruir desde índice vacío y evaluar (`bun run lab:rebuild && bun run lab:eval`)

En la terminal, muestra el flujo de reconstrucción y evaluación del laboratorio (`lab/`):

```
bun run lab:rebuild
bun run lab:eval
```

Señala que:

- `rebuildFromEmptyIndex` vacía la tabla (`TRUNCATE`), lee los 10 Markdown de `lab/corpus/sources/`, genera los 20 fragmentos y sus vectores de `768` dimensiones, los inserta en `vector(768)` con índice HNSW y comprueba el conteo de vuelta.
- `bun run lab:eval` ejecuta las **20 preguntas de test (14 respondibles y 6 sin respuesta)** y guarda la tabla en `lab/eval/resultados-laboratorio.md` (`16/20` en RAG básico: `10/14` respondibles y `6/6` rechazos correctos).

75–80 min · Enunciado publicado de S10 y transición a CRAG (Diapositiva 12)

Abre `material/ejercicios/S10-enunciado.md` y repasa los requisitos de entrega:

- **8 o más documentos ficticios** en Markdown/TXT y reconstrucción completa desde archivos en bruto e índice vacío.
- **PostgreSQL con `pgvector`**, columna `vector(768)` e **índice HNSW**, con modelos configurables por variables (`GEN_MODEL_ID`, `EMBEDDING_MODEL_ID`, `DATABASE_URL`).
- **Control de puntuación previo a generar** calibrado con validación separada del test, bloque de datos no fiable y citas verificables.
- **Evaluación ejecutada de 20 preguntas (14 respondibles y 6 sin respuesta)** presentando los resultados (`X/14` y `Y/6`, fragmentos, puntuaciones, respuestas y fuentes por caso). Recuerda que una evaluación solo preparada y sin números no equivale a completar la evaluación solicitada.

80–90 min · Preguntas y cierre (Diapositivas 13 a 15)

- Si preguntan cómo se relacionan el demostrador web, el laboratorio `lab/` y el modo `demo` offline, proyecta la **Diapositiva 13** (tabla de los tres perfiles).
- Si preguntan cómo recuperar el entorno local sin red, proyecta la **Diapositiva 14** (`bun run preflight && bun run dev`).
- Cierra en la **Diapositiva 15** con el QR (recuerda que evalúa S9) y anuncia el paso a **CRAG el martes 27 de octubre (S11)**.